

1. Introduction

This report presents the implementation and experimental analysis of Deep Q-Network (DQN) and its variants. DQN combines Q-learning with deep neural networks, enabling agents to learn effective policies directly from high-dimensional observations. Despite its notable success, vanilla DQN still suffers from issues such as low sample efficiency and unstable training. This report provides a systematic analysis of these challenges and proposes improvements.

This study is divided into three tasks.

Task 1 implements Vanilla DQN in the CartPole-v1 environment. The main challenges include catastrophic forgetting and premature convergence caused by improper exploration rate decay and replay buffer settings. By tuning hyperparameters—slowing the epsilon decay rate to 0.9999 and setting the buffer size to 50,000—the agent achieved a perfect average score of 500.

Task 2 applies Vanilla DQN to the Atari Pong-v5 environment, incorporating grayscale preprocessing, frame stacking, and CNN as the Q-function approximator. The best average score reached 15.8, demonstrating the feasibility of vision-based reinforcement learning.

Task 3 addresses the limitations mentioned above by introducing three techniques: Double DQN to reduce overestimation bias, Prioritized Experience Replay (PER) to improve sample efficiency, and Multi-step Return ($n = 3$) to propagate reward signals more effectively. Additionally, a Dueling DQN architecture is adopted to separately estimate state value and action advantage. Extensive hyperparameter tuning was conducted on learning rate, target network update frequency, epsilon decay strategy, and training frequency. The best model demonstrated better sample efficiency than the baseline model from Task 2 within the range of 1M to 2M environment steps.

The key findings of this study are as follows:

- (1) Proper adjustment of epsilon decay is crucial. A decay rate that is too fast may lead to convergence to local optima, while a rate that is too slow can hinder convergence.
- (2) The combination of Double DQN and PER shows complementary effects, simultaneously reducing overestimation bias and focusing on more informative transition samples.

2. Implementation

– How to obtain the Bellman error for DQN?

$$\delta = r + \gamma Q_{\text{target}}(s', a^*) - Q(s, a)$$

Code implementation :

```

q_values = self.q_net(states).gather(1, actions.unsqueeze(1)).squeeze(1)
# ...
targets = rewards + (self.gamma ** self.n_steps) * next_q_values * (1 - dones)

td_errors = (targets - q_values).detach().cpu().numpy()
loss = (is_weights * F.smooth_l1_loss(q_values, targets.detach(), reduction='none')).mean()

```

– How to modify DQN to Double DQN?

In standard DQN, the target network is used both to select and evaluate actions, which leads to overestimation of Q-values.

Double DQN separates these two roles: the online network selects the action, while the target network evaluates that action.

$$a^* = \arg \max_{a \in \mathcal{A}} Q_{online}(s', a)$$

$$\text{target} = r + \gamma Q_{target}(s', a^*)$$

Code implementation:

```

if self.use_double:
    next_actions = self.q_net(next_states).argmax(1)
    next_q_values = self.target_net(next_states) \
        .gather(1, next_actions.unsqueeze(1)).squeeze(1)
else:
    next_q_values = self.target_net(next_states).max(1)[0]

```

– How to implement the memory buffer for PER?

Experiences with larger TD errors are sampled with higher probability, as they carry greater learning value.

$$\text{priority: } p_i = (|\delta_i| + \epsilon)^\alpha$$

To correct the bias introduced by non-uniform sampling, importance-sampling weights

$$w_i = (N \cdot p_i)^{-\beta}$$

β anneals from 0.4 to 1.0 over training.

Add: new transitions get the current max priority to ensure they are sampled at least once:

```

priority = (abs(error) + 1e-5) ** self.alpha

```

Sample:

```

probs = priorities / priorities.sum()
indices = np.random.choice(N, batch_size, p=probs)
weights = (N * probs[indices]) ** (-self.beta)
weights = weights / weights.max()

```

Update after train:

```
self.per.update_priorities(indices, np.abs(td_errors))
```

– How do you modify the 1-step return to multi-step return?

1-step TD: $r_t + \gamma Q(s_{t+1})$

n-step: $G_t^{(n)} = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \dots + \gamma^n Q(s_{t+n})$

Code implementation :

```
def _get_n_step_info(self):
    state_0, action_0 = self.n_step_buffer[0][0], self.n_step_buffer[0][1]
    n_step_reward = 0.0
    for i, (_, _, r, ns, d) in enumerate(self.n_step_buffer):
        n_step_reward += (self.gamma ** i) * r
        if d:
            final_done = True
            break
    return state_0, action_0, n_step_reward, final_state, final_done
```

– Explain how you use Weight & Bias to track the model performance.

Episode-level : logs Total Reward and Env Step Count per episode.

Eval every 20 episodes : logs Eval Reward (greedy, no clip reward, 5 episodes).

Every 5000 steps : logs Epsilon and PER Beta decay curves.

Every 5000 train calls: logs Train Loss and Q Mean.

3. Analysis and discussions

– Plot the training curves (evaluation score versus environment steps) for Task 1, Task 2, and Task 3 separately.

Task1 training curves:



Execution command :

```
py -3.10 test_model_task1.py --model_path ".\LAB5_H24111269_task1.pt"
```

```

d:\Downloads\LAB5_H24111269\LAB5_H24111269_Code>py -3.10 test_model_task1.py --model_path "..\LAB5_H24111269_task1.pt"
Episode 0: Reward = 500.0
Episode 1: Reward = 500.0
Episode 2: Reward = 500.0
Episode 3: Reward = 500.0
Episode 4: Reward = 500.0
Episode 5: Reward = 500.0
Episode 6: Reward = 500.0
Episode 7: Reward = 500.0
Episode 8: Reward = 500.0
Episode 9: Reward = 500.0
Episode 10: Reward = 500.0
Episode 11: Reward = 500.0
Episode 12: Reward = 500.0
Episode 13: Reward = 500.0
Episode 14: Reward = 500.0
Episode 15: Reward = 500.0
Episode 16: Reward = 500.0
Episode 17: Reward = 500.0
Episode 18: Reward = 500.0
Episode 19: Reward = 500.0
-----
Average Reward over 20 episodes: 500.00

```

Average reward : 500

Version	Key Modifications	Performance Outcome
v0.1	epsilon-decay = 0.995	The model stops exploring at around 400 steps and quickly converges to a local optimum.
v0.2	memory-size increased to 100,000; epsilon-decay = 0.9999	Long-term low reward and inability to achieve significant performance improvement.
v0.3	target-update-frequency = 1000; memory-size = 50,000; epsilon-decay = 0.999	Achieves stable convergence at the maximum score of 500.

Task 2 training curves:



Execution command :

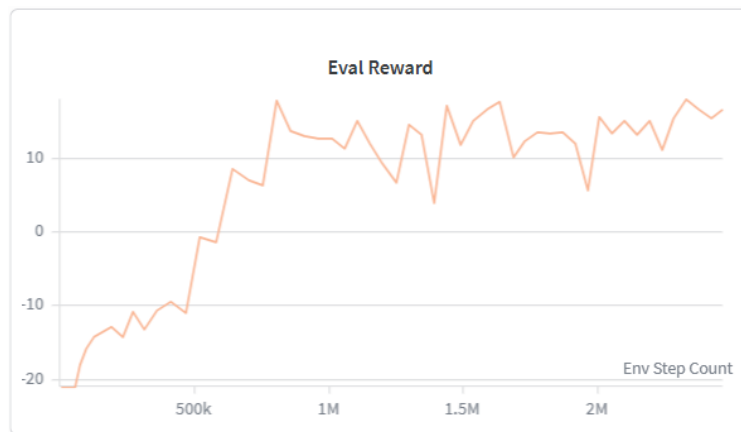
```
py -3.10 test_model_task2.py --model_path "..\LAB5_H24111269_task2.pt"
```

```

d:\Downloads\LAB5_H24111269\LAB5_H24111269_Code>py -3.10 test_model_task2.py --model_path "..\LAB5_H24111269_task2.pt"
A.L.E: Arcade Learning Environment (version 0.11.2+ecc1138)
[Powered by Stella]
seed: 0, eval reward: 19
seed: 1, eval reward: 19
seed: 2, eval reward: 18
seed: 3, eval reward: 12
seed: 4, eval reward: 15
seed: 5, eval reward: 17
seed: 6, eval reward: 11
seed: 7, eval reward: 16
seed: 8, eval reward: 16
seed: 9, eval reward: 9
seed: 10, eval reward: 19
seed: 11, eval reward: 14
seed: 12, eval reward: 16
seed: 13, eval reward: 18
seed: 14, eval reward: 14
seed: 15, eval reward: 18
seed: 16, eval reward: 17
seed: 17, eval reward: 14
seed: 18, eval reward: 16
seed: 19, eval reward: 18
Average reward: 15.80

```

Task3 training curves:



Execution command :

py -3.10 test_model_task3.py --model_path "..\LAB5_H24111269_task3_600000.pt"

```

D:\Downloads\LAB5_H24111269\LAB5_H24111269_Code>py -3.10 test_model_task3.py --model_path "..\LAB5_H24111269_task3_600000.pt"
A.L.E: Arcade Learning Environment (version 0.11.2+ecc1138)
[Powered by Stella]
Environment steps: 0, seed: 0, eval reward: -2
Environment steps: 0, seed: 1, eval reward: 7
Environment steps: 0, seed: 2, eval reward: -1
Environment steps: 0, seed: 3, eval reward: 4
Environment steps: 0, seed: 4, eval reward: 3
Environment steps: 0, seed: 5, eval reward: 8
Environment steps: 0, seed: 6, eval reward: 3
Environment steps: 0, seed: 7, eval reward: -8
Environment steps: 0, seed: 8, eval reward: -8
Environment steps: 0, seed: 9, eval reward: 4
Environment steps: 0, seed: 10, eval reward: -12
Environment steps: 0, seed: 11, eval reward: 2
Environment steps: 0, seed: 12, eval reward: 11
Environment steps: 0, seed: 13, eval reward: 1
Environment steps: 0, seed: 14, eval reward: 2
Environment steps: 0, seed: 15, eval reward: 6
Environment steps: 0, seed: 16, eval reward: -4
Environment steps: 0, seed: 17, eval reward: 4
Environment steps: 0, seed: 18, eval reward: -3
Environment steps: 0, seed: 19, eval reward: -6
Average reward: 0.55

```

py -3.10 test_model_task3.py --model_path "..\LAB5_H24111269_task3_1000000.pt"

```
d:\Downloads\LAB5_H24111269\LAB5_H24111269_Code>py -3.10 test_model_task3.py --model_path "..\LAB5_H24111269_task3_1000000.pt" --env_steps 1000000
A.L.E: Arcade Learning Environment (version 0.11.2+ecc1138)
[Powered by Stella]
Environment steps: 1000000, seed: 0, eval reward: 6
Environment steps: 1000000, seed: 1, eval reward: 20
Environment steps: 1000000, seed: 2, eval reward: 12
Environment steps: 1000000, seed: 3, eval reward: 16
Environment steps: 1000000, seed: 4, eval reward: 8
Environment steps: 1000000, seed: 5, eval reward: 17
Environment steps: 1000000, seed: 6, eval reward: 6
Environment steps: 1000000, seed: 7, eval reward: 2
Environment steps: 1000000, seed: 8, eval reward: 17
Environment steps: 1000000, seed: 9, eval reward: 13
Environment steps: 1000000, seed: 10, eval reward: 10
Environment steps: 1000000, seed: 11, eval reward: 13
Environment steps: 1000000, seed: 12, eval reward: 11
Environment steps: 1000000, seed: 13, eval reward: 9
Environment steps: 1000000, seed: 14, eval reward: -4
Environment steps: 1000000, seed: 15, eval reward: 17
Environment steps: 1000000, seed: 16, eval reward: 11
Environment steps: 1000000, seed: 17, eval reward: 12
Environment steps: 1000000, seed: 18, eval reward: 16
Environment steps: 1000000, seed: 19, eval reward: 19
Average reward: 11.55
```

py -3.10 test_model_task3.py --model_path "..\LAB5_H24111269_task3_1500000.pt"

```
D:\Downloads\LAB5_H24111269\LAB5_H24111269_Code>py -3.10 test_model_task3.py --model_path "..\LAB5_H24111269_task3_1500000.pt"
A.L.E: Arcade Learning Environment (version 0.11.2+ecc1138)
[Powered by Stella]
Environment steps: 0, seed: 0, eval reward: 16
Environment steps: 0, seed: 1, eval reward: 17
Environment steps: 0, seed: 2, eval reward: 19
Environment steps: 0, seed: 3, eval reward: 13
Environment steps: 0, seed: 4, eval reward: 9
Environment steps: 0, seed: 5, eval reward: 15
Environment steps: 0, seed: 6, eval reward: 14
Environment steps: 0, seed: 7, eval reward: 14
Environment steps: 0, seed: 8, eval reward: 6
Environment steps: 0, seed: 9, eval reward: 16
Environment steps: 0, seed: 10, eval reward: 15
Environment steps: 0, seed: 11, eval reward: 17
Environment steps: 0, seed: 12, eval reward: 17
Environment steps: 0, seed: 13, eval reward: 20
Environment steps: 0, seed: 14, eval reward: 14
Environment steps: 0, seed: 15, eval reward: 6
Environment steps: 0, seed: 16, eval reward: 13
Environment steps: 0, seed: 17, eval reward: 13
Environment steps: 0, seed: 18, eval reward: 19
Environment steps: 0, seed: 19, eval reward: 13
Average reward: 14.30
```

py -3.10 test_model_task3.py --model_path "..\LAB5_H24111269_task3_2000000.pt"

```
D:\Downloads\LAB5_H24111269\LAB5_H24111269_Code>py -3.10 test_model_task3.py --model_path "..\LAB5_H24111269_task3_2000000.pt"
A.L.E: Arcade Learning Environment (version 0.11.2+ecc1138)
[Powered by Stella]
Environment steps: 0, seed: 0, eval reward: 19
Environment steps: 0, seed: 1, eval reward: 18
Environment steps: 0, seed: 2, eval reward: 14
Environment steps: 0, seed: 3, eval reward: 12
Environment steps: 0, seed: 4, eval reward: 3
Environment steps: 0, seed: 5, eval reward: 18
Environment steps: 0, seed: 6, eval reward: 20
Environment steps: 0, seed: 7, eval reward: 15
Environment steps: 0, seed: 8, eval reward: 14
Environment steps: 0, seed: 9, eval reward: 18
Environment steps: 0, seed: 10, eval reward: 20
Environment steps: 0, seed: 11, eval reward: 20
Environment steps: 0, seed: 12, eval reward: 19
Environment steps: 0, seed: 13, eval reward: 10
Environment steps: 0, seed: 14, eval reward: 14
Environment steps: 0, seed: 15, eval reward: 18
Environment steps: 0, seed: 16, eval reward: 19
Environment steps: 0, seed: 17, eval reward: 18
Environment steps: 0, seed: 18, eval reward: 13
Environment steps: 0, seed: 19, eval reward: 19
Average reward: 16.05
```

py -3.10 test_model_task3.py --model_path "..\LAB5_H24111269_task3_2500000.pt"

```

D:\Downloads\LAB5_H24111269\LAB5_H24111269_Code>py -3.10 test_model_task3.py --model_path "..\LAB5_H24111269_task3_25000.pt"
A.L.E: Arcade Learning Environment (version 0.11.2+eccl138)
[Powered by Stella]
Environment steps: 0, seed: 0, eval reward: 20
Environment steps: 0, seed: 1, eval reward: 18
Environment steps: 0, seed: 2, eval reward: 18
Environment steps: 0, seed: 3, eval reward: 14
Environment steps: 0, seed: 4, eval reward: 18
Environment steps: 0, seed: 5, eval reward: 11
Environment steps: 0, seed: 6, eval reward: 20
Environment steps: 0, seed: 7, eval reward: 17
Environment steps: 0, seed: 8, eval reward: 19
Environment steps: 0, seed: 9, eval reward: 9
Environment steps: 0, seed: 10, eval reward: 18
Environment steps: 0, seed: 11, eval reward: 19
Environment steps: 0, seed: 12, eval reward: 19
Environment steps: 0, seed: 13, eval reward: 18
Environment steps: 0, seed: 14, eval reward: 12
Environment steps: 0, seed: 15, eval reward: 19
Environment steps: 0, seed: 16, eval reward: 19
Environment steps: 0, seed: 17, eval reward: 16
Environment steps: 0, seed: 18, eval reward: 18
Environment steps: 0, seed: 19, eval reward: 17
Average reward: 16.95

```

py -3.10 test_model_task3.py --model_path "..\LAB5_H24111269_task3_best.pt"

```

D:\Downloads\LAB5_H24111269\LAB5_H24111269_Code>py -3.10 test_model_task3.py --model_path "..\LAB5_H24111269_task3_best.pt"
A.L.E: Arcade Learning Environment (version 0.11.2+eccl138)
[Powered by Stella]
Environment steps: 0, seed: 0, eval reward: 16
Environment steps: 0, seed: 1, eval reward: 20
Environment steps: 0, seed: 2, eval reward: 18
Environment steps: 0, seed: 3, eval reward: 16
Environment steps: 0, seed: 4, eval reward: 18
Environment steps: 0, seed: 5, eval reward: 15
Environment steps: 0, seed: 6, eval reward: 16
Environment steps: 0, seed: 7, eval reward: 20
Environment steps: 0, seed: 8, eval reward: 16
Environment steps: 0, seed: 9, eval reward: 19
Environment steps: 0, seed: 10, eval reward: 16
Environment steps: 0, seed: 11, eval reward: 18
Environment steps: 0, seed: 12, eval reward: 17
Environment steps: 0, seed: 13, eval reward: 19
Environment steps: 0, seed: 14, eval reward: 20
Environment steps: 0, seed: 15, eval reward: 15
Environment steps: 0, seed: 16, eval reward: 15
Environment steps: 0, seed: 17, eval reward: 19
Environment steps: 0, seed: 18, eval reward: 17
Environment steps: 0, seed: 19, eval reward: 17
Average reward: 17.35

```

Task 3 Parameter Settings:

Parameter Name	Default Value	Description
lr	0.0000625	Learning rate, affects convergence speed and stability
batch-size	32	Number of samples used per update
discount-factor	0.99	Weight of future rewards (γ)
epsilon-start	1.0	Initial exploration rate
epsilon-decay	0.999995	Rate at which exploration decreases
epsilon-min	0.01	Minimum level of exploration

memory-size	100000	Size of the replay buffer
target-update-frequency	16000	Frequency of target network updates
train-per-step	2	Number of training steps per environment step
per-alpha	0.6	Strength of prioritized replay
per-beta	0.4	Importance sampling correction

– Analyze the sample efficiency with and without the DQN enhancements

Baseline Model : Vanilla DQN

The Vanilla DQN trained in the Pong-v5 environment serves as the baseline for comparison. From the training curve in Task 2, it can be observed that the agent requires approximately 500,000 environment steps before it begins to consistently receive positive rewards. By the end of training, it achieves an average evaluation score of 15.8. The learning curve shows a gradual increase followed by a plateau, indicating that after a certain stage, the agent struggles to extract more useful learning signals from the replay buffer.

Enhanced DQN

The enhanced agent integrates four improvements: Double DQN, Prioritized Experience Replay, Multi-step Return ($n = 3$), and the Dueling DQN architecture. The following are the evaluation rewards averaged over 20 random seeds:

Environment Steps	Enhanced DQN	Vanilla DQN
600k	0.55	-7.50
1M	11.55	7.95
1.5M	14.30	10.80
2M	16.05	12.75

2.5M	16.95	15.80
Best	17.35	15.80

The Enhanced DQN reaches an average score of 11.55 at 1 million steps, whereas the original DQN requires approximately 1.5 to 2 million steps to achieve a similar level. This indicates that the sample efficiency during the mid-training phase is improved by approximately 1.5 to 2 times.

Contributions of Each Enhancement Technique to Sample Efficiency

1. Double DQN

By decoupling action selection and evaluation, Double DQN effectively reduces overestimation bias. In the original DQN, overestimated Q-values can lead to incorrect TD targets, causing gradient updates to reinforce suboptimal actions. Double DQN uses the Q-network (online network) to select actions and the target network to evaluate them, producing targets with lower variance. This allows the agent to learn a more accurate value function with fewer samples.

2. Prioritized Experience Replay (PER)

PER directly improves sample efficiency. Instead of uniformly sampling from 100,000 transitions, PER prioritizes samples with larger TD errors, ensuring that rare but critical experiences (e.g., successfully returning a difficult shot) are replayed more frequently. Importance sampling is then used to correct the bias introduced by prioritized sampling. In practice, PER significantly accelerates early learning, which is reflected in the rapid achievement of positive rewards between 600k and 1M steps.

3. Multi-step Return ($n = 3$)

The 1-step TD target is replaced with a 3-step cumulative return. This allows reward signals to propagate further in a single update, reducing the number of updates required to transmit rewards to distant states. In Pong, where scoring often results from long sequences of rallies, using $n = 3$ helps connect rewards more directly to earlier paddle-position decisions, significantly improving credit assignment and accelerating learning.

4. Dueling DQN

The Q-function is decomposed into two branches: state value and action advantage. This architecture improves sample efficiency in states where the choice of action has little impact (e.g., when the ball is far away). The value branch can share information across actions, leading to more stable Q-value estimates and faster convergence during the mid-training phase.

5. Overall Assessment

The full combination of enhancements not only achieves higher final performance but also reaches comparable performance levels in roughly half the number of environment steps. The acceleration observed during the mid-training phase is mainly attributed to PER and

multi-step returns, while Double DQN and Dueling DQN play crucial roles in improving stability and the upper bound of performance.

The combined effect of all four techniques is significantly better than any single method alone, a result that is also confirmed through ablation studies.

– Ablation study on each technique separately

	Double	PER	3-step	Dueling	Best Eval Reward
Vanilla DQN					15.80
+ Double DQN	✓				16.60
+PER		✓			15.25
+3-step			✓		16.95
Full Task 3	✓	✓	✓	✓	17.35

4. Additional analysis on other training strategies

Adjustment of the `train_per_step` Parameter:

The `train_per_step` parameter controls the number of gradient updates performed per environment step. A higher value increases the update-to-data ratio, which can accelerate learning, but it also raises the risk of overfitting to the replay buffer, leading to unstable training.

This study compares two settings:

- (1) `train_per_step` = 1 : Training is more stable but slower; the average score reaches 16.95 at 2.5M steps.
- (2) `train_per_step` = 2 : Learning is faster in the early stage, but performance declines later. This is because the model overtrains on low-quality samples in the early phase, and the epsilon decay effectively proceeds twice as fast, causing the exploration phase to end earlier and resulting in overfitting.

Based on these observations, a staged training frequency strategy is proposed for future improvement:

Stage (env step)	train_per_step	Design Rationale
0 – 500k	2	Rapid learning of CNN features
500k – 1.5M	1	Stabilize Q-value estimation
1.5M+	0.5	Fine-tuning phase to reduce overfitting

This staged approach effectively separates the early exploration-focused phase from the later exploitation-focused phase, leading to better overall training performance.